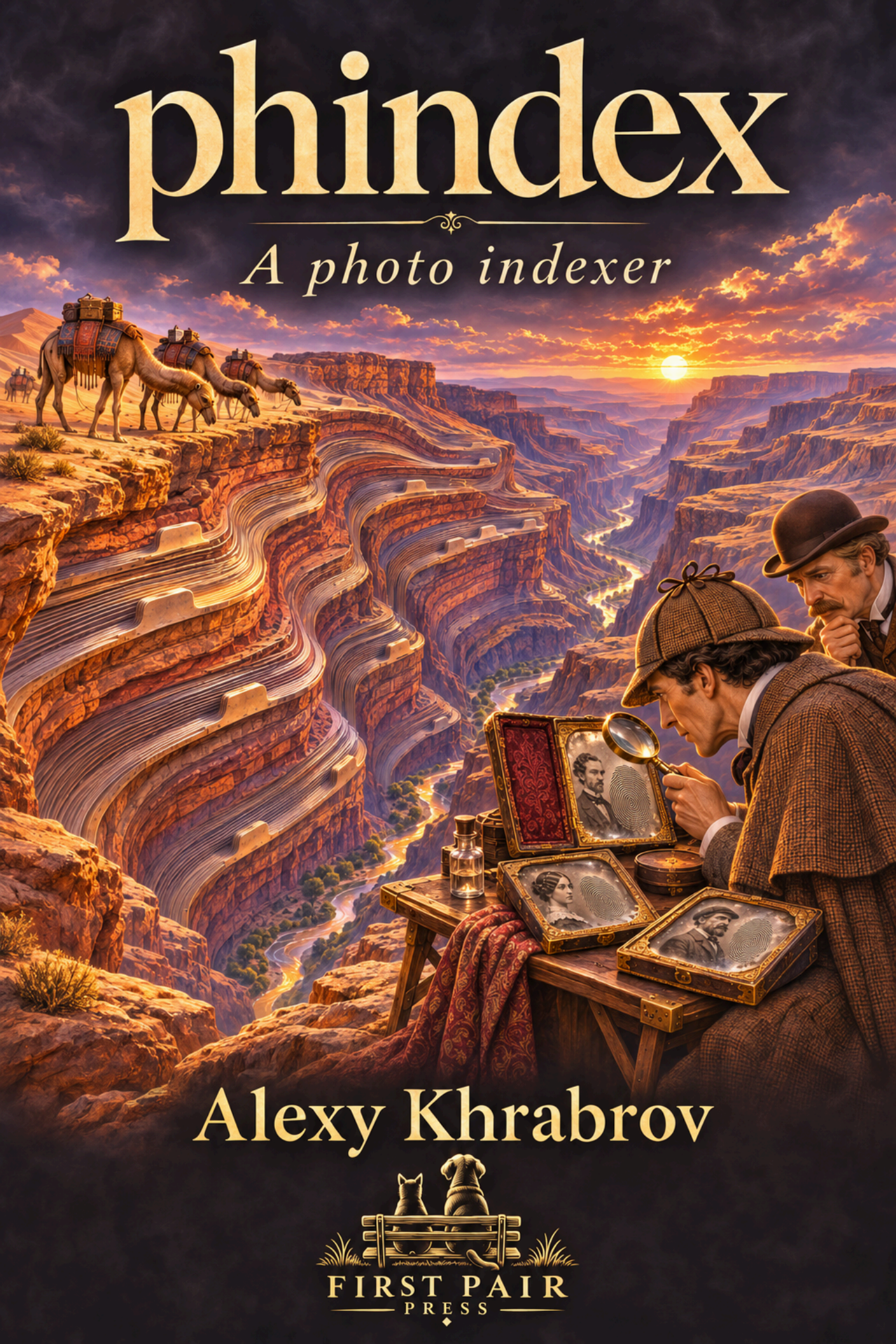
phindex

A photo indexer



Alexy Khrabrov

FIRST PAIR
PRESS

phindex

An index of photo identities that is cheap to rerun

Alexy Khrabrov

2026-11-11

Version 0.7.0-1b6e940a
Source commit 1b6e940afe9aa284c81da03d9687581eb0f5ee13

First Pair Press
firstpair.press

Contents

About this manual 4
What phindex is 5
The identity columns 5
Image data: an identity that survives metadata edits 6
Block marks, skeletons and earlier contents 7
Three states, and never a match on failure 8
Where it fits 8
What it is not 9
Building and running 10
phindex (Rust) 10
phindex-ox (stock OCaml and OxCaml) 10
The command line 11
What it prints 11
The launch header 12
The runner script 12
The schema 14
files 14
skeletons 14
versions 15
metadata 15
dirs 15
runs 16
Incremental behaviour 17
What a rerun reads 17
The revision stamp 17
Upgrading from the previous revision 17
Skeletons from a store that already has them 18
Rows for copies, from the other tree's index 18
The quick walk 19
In-place rewrites and `--full-stat` 19
`--only` 20
Moves and renames 20
Gone 20
Commits, interruptions and resuming 21
A file that changes during the run 21
Two implementations, one index 22
phindex: Rust 22
phindex-ox: OCaml, stock and OxCaml 22
The comparison 22
Files over 1 GiB 23
Which to use 23
Performance and tuning on hard disks 24
Where the time goes 24
When workers help 24

- When the disks limit ... 24
- When the enclosure limits ... 24
- Reading in path order ... 25
- Choosing `--workers` ... 25
- Small files lower MB/s, not files per second ... 25
- Memory ... 26

Queries ... 27
- Paths under a root ... 27
- Containment between two roots ... 27
- Metadata-only differences ... 27
- Lightroom catalog matching ... 28
- Copying an index ... 28
- Housekeeping ... 29

Aligning two trees ... 30
- Units ... 30
- How a unit is matched ... 30
- Statuses and flags ... 31
- Right-only units ... 32
- The report ... 32
- Progress ... 32
- From alignment to raw comparison ... 33
- A worked example ... 33

Renaming a tree to match another ... 35
- The plan ... 35
- Units that are left alone ... 35
- Applying it ... 36
- Undoing it ... 36
- What it does not do ... 36

Setting aside what another tree lacks ... 37
- What `diff` lists ... 37
- Moving a list ... 38
- Moving one file or one folder ... 38

Feeding dngc ... 39

Testing ... 40

Licences ... 42

Reference ... 43

About this manual

This manual is by Alexy Khrabrov, with Claude as co-author. The ideas and decisions are Alexy's; Claude wrote the text and the code with him.

It describes phindex 0.7.0 as of 2026-11-10: the Rust binary `phindex` and its OCaml counterpart `phindex-ox`, which runs under stock OCaml 5 and under OxCaml. phindex is licensed under the MIT License; the binaries link code under other licences, as the chapter on licences explains. Its source is at github.com/firstpair/phindex.

0.7.0 adds `phindex adopt`, which gives files copied from another indexed tree their rows from that tree's index, without opening them.

0.6.0 added `phindex diff`, which lists the files of one tree that are at no path of another, and `phindex mv`, which moves a listed set, a file or a folder with the index following: only the index rows by default, the files too with `--files`. Together they set aside what another tree lacks.

0.5.0 added `phindex conform`, which renames the aligned files of one tree to the names they have in another, in place, with a plan to review first, a journal and an undo. It is the first command that changes an indexed tree, and it changes only names. It also adds `import-skeletons`, which fills an index from a skeleton store without opening a file. The revision stamp is unchanged, so an index written by 0.4.0 is current.

0.4.0 added one identity and a way back from metadata edits. Every file gets `block_marks`, phidentify's identity of a file's large blocks wherever they sit, and the index keeps a **skeleton** of every content it reads and the SHA-256 of each path's earlier contents. With those, `phindex restore` rebuilds a file as it was before an editor wrote into it, from the edited file. The revision stamp gains `; block-marks/1`. An index written by 0.2.0 or 0.3.0 is upgraded at the next run from each file's structures, without reading any file in full; the earlier one-step upgrade, for indexes written before `image_data`, is gone, and such an index is read again in full.

Every example in this manual uses synthetic files or generic paths such as `/archive`, `/backup`, `/Volumes/Archive` and `/Volumes/Backup`. Nothing here depends on a particular photo library, disk or catalog.

What phindex is

phindex builds, and keeps up to date, an index of every regular file under a directory tree, in one SQLite file. The tree might be a photo archive, a backup volume or a disk that came back from a trip. For each file the index records where it is, its size, mtime, inode and device, and a set of **identities** computed by phidentify: the whole-file SHA-256, the DNG encoded raw blocks, the Lightroom Classic original fingerprint, the metadata record digest, the tiffgraph grammar with the preserved digest, and the image data of camera raws, TIFFs and JPEGs that are not DNGs. All of them come from one identification of the file: phidentify reads it into memory once and takes every identity it can from those bytes, and checks that the file did not change while it worked.

The index exists to answer archive questions without reading the archive again. Which of these files have a byte-identical copy on the backup? Which DNGs differ from their copy only in their XMP packet? Which file on disk is the original a Lightroom catalog entry recorded? Each of those is an SQL query against the index, and each takes seconds, where reading a multi-terabyte tree takes days.

So the index has to be cheap to keep current. Most of this manual is about that: a rerun reads only files that are new, changed or unfinished, finds changed directories from their mtimes without listing the rest, carries identities across moves and renames, resumes after an interruption where it stopped, and, when a new identity is added, upgrades the existing rows while reading as few files as it can.

Once two trees are indexed, `phindex align` accounts for every file of one in the other by identity rather than by name, so copies that were moved, renamed or had their metadata edited are still found. It reads only the two indexes, never the files.

The identity columns

Every identity has its own column in the `files` table, and every identity that can fail has a matching `_err` column for the reason.

Column	Identity	Attempted for	Equal values mean
`sha256`	whole-file SHA-256	every file	every byte is equal
`dng_blocks`, `dng_blocks_err`	DNG encoded raw blocks	files with a TIFF header	same encoded primary raw data; metadata may differ
`lr_digest`, `lr_err`	Lightroom Classic original fingerprint	files that parse as DNGs	Lightroom Classic's matcher would pair them
`md_digest`, `md_err`	exiftool-rs metadata record, file location left out	files whose extension the engine reads	same metadata record from the same engine
`dngsem_profile`, `dngsem_err`	strictest tiffgraph grammar that accepts the file	every file	the same grammar applies
`preserved`, `preserved_err`	Semantic Unison's `SDE/0-preserved` digest	files a grammar accepted	every byte outside the XMP packet is equal, at the same size and layout
`image_data`, `image_err`	stored image bytes of a camera raw, TIFF or JPEG that is not a DNG	every file	same image bytes in the same order; metadata may differ
`block_marks`, `block_marks_err`	the file's large blocks, by length and by a hash of each block's two ends	every file	same blocks, wherever they sit: the same picture with the same previews, maker

Column	Identity	Attempted for	Equal values mean
			notes and profiles; metadata may differ
`ids_rev`	the revision stamp of the identity definitions	every identified file	the identities were computed by the same definitions

phidentify's manual defines each identity exactly: what goes into it, what equality proves and what it does not, and what it costs. This manual does not repeat those definitions. In short:

- **`sha256`** is the only identity that proves two files are the same bytes. Files over 1 GiB get a streamed SHA-256, their metadata record and the tiffgraph walk from disk, and nothing that needs the bytes in memory.
- **`dng_blocks`** hashes the encoded raw image data of a DNG and its encoding tags, without offsets, XMP, previews or maker notes. A keyword edit cannot change it; a flipped raw byte always does.
- **`lr_digest`** is phidentify's reimplementation of the fingerprint Lightroom Classic keeps for every original, in upper-case hex as Classic stores it. It is computed only for the DNG strata phidentify supports; for anything else it is an error, never a guess.
- **`md_digest`** hashes the ExifTool-shaped record that dng-compare's exiftool-rs engine writes, without the members that say where the file sits (its name, directory and file dates). The engine's own version stamp stays in the hash, so records from different engines never compare equal. Only files whose extension the engine reads are attempted; any other file, such as a Capture One `.cop` cache proxy or an `.ini`, keeps both `md_digest` and `md_err` empty and has no record. An index written before that rule is corrected at the start of the next update: those rows lose their digest, error and record, without a read and without a new revision stamp.
- **`dngsem_profile`** names the strictest of the grammars `dng-xmp/0`, `dng-xmp/1` and `dng-xmp/2` under which tiffgraph found the primary XMP packet and checked that nothing else the file declares lies inside it.
- **`preserved`** hashes every byte outside that packet, with the file size and the packet's position bound in. Two files with equal `preserved` values under the same `dngsem_profile` can differ only inside the packet.
- **`image_data`** hashes only the stored image bytes of a file that is not a DNG, as the next section explains.

Image data: an identity that survives metadata edits

`dng_blocks` lets two DNGs agree although their metadata differs. Until 0.2.0 nothing did the same for the other files of an archive: a NEF, an ARW or a JPEG whose EXIF had been edited agreed with its original only on the Lightroom fingerprint, which phidentify computes only for DNGs, so for those files nothing agreed at all. `image_data`, phidentify's sixth identity, fills that gap. It hashes the bytes that hold the picture and leaves out the bytes that describe it:

- **TIFF and the TIFF-based raws** (NEF, CR2, ARW, 3FR, PEF, ORF and others, and BigTIFF): the strip or tile payloads of every IFD, in the main chain and in its SubIFDs, IFD by IFD in walk order, each with its byte counts. The tag tables, EXIF and maker notes, which say where things are and what they mean, are not hashed; only what the strip and tile offsets point to is.
- **Panasonic RW2**: its raw data is named by RawDataOffset (tag 280), with no byte counts, and runs to the end of the file; that span is taken.
- **Fujifilm RAF**: the CFA section whose offset and length the RAF header gives.
- **JPEG**: the primary image's frame header, tables and entropy-coded scans, that is every segment from SOI to EOI except the APPn segments (EXIF, XMP, ICC profiles, embedded thumbnails) and comments. Anything after EOI, such as a trailer some cameras and phones append, is left out.

Two files have equal `image_data` exactly when those bytes are equal, in the same order. A metadata edit that leaves the image bytes in place keeps the value; re-encoding, recompressing or rewriting a preview strip changes it. phidentify checked this on real files: shifting the capture time of a TIFF, an ARW, an RW2 and a JPEG with ExifTool changed each one's whole-file SHA-256 and left its `image_data` equal. ExifTool cannot write 3FR, so that format was not part of the test.

Some files are refused, with a reason in `image_err`:

- **DNGs**, with `DNG: the raw-block identity applies instead`. A DNG's strips include the previews Lightroom regenerates when it saves metadata, so an image-data digest of a DNG would change with a metadata edit; `dng_blocks` hashes only the primary raw data and is the right identity for DNGs.
- **Files that are not a TIFF-family file or JPEG**, with `not a TIFF-family file or JPEG`: CR3, HEIC and other ISO-media containers, videos, sidecars, text. The decision is made from the first 16 bytes, which is what makes the upgrade described under *Incremental behaviour* cheap.

Why it matters: the commonest edit to a set of camera files is a capture-time shift, made when the camera clock was wrong or in another time zone. It rewrites a few bytes of EXIF in every file, so every whole-file SHA-256 changes, and a byte-for-byte comparison against an unshifted copy says that nothing is backed up. With `image_data` the shifted copy and the original still agree, and `phindex align` can pair them and say that only their metadata differs.

Block marks, skeletons and earlier contents

An editor that writes a rating, a caption or develop settings into a photo rewrites the file's small structures and carries its large blocks over unchanged: the raw strips or tiles, the embedded previews, the maker notes, the profiles. phidentify cuts a file along those lines, by the rules of DNG Compare's archive format, into **blocks** and the **skeleton**, which is every byte outside the blocks. phindex stores three things from that cut:

- **`block_marks`**, an identity: equal values mean the two files hold blocks of the same lengths with the same first and last 4,096 bytes. It applies to DNGs, other TIFF-based raws and JPEGs, and it survives a metadata edit. On 249 real files compared before and after years of edits in Lightroom Classic and Photo Mechanic, it was equal in all 249. It is a sampled identity: a strong hint, and not proof of equal bytes.
- **A skeleton per content**, in the `skeletons` table by SHA-256: the packed bytes outside the blocks, and a record of where each block goes. On real files that is about 5 KB for a DNG and 10 KB for a JPEG. A file of another format up to 1 MiB, such as an XMP sidecar, is stored whole as its own skeleton; a larger one, such as a video, has none.
- **Earlier contents per path**, in the `versions` table: when a run finds that a path's content has changed, it records the SHA-256 the path had before.

Together they make an indexed tree its own safety net against metadata edits. The file as it was is the old skeleton with the blocks taken from the file as it is now:

```
phindex versions --db archive.sqlite --path /archive/2024/IMG_0042.dng
phindex restore  --db archive.sqlite --path /archive/2024/IMG_0042.dng --out /tmp/
IMG_0042-before.dng
phindex restore  --db archive.sqlite --sha256 5b8de6f0… --out /tmp/as-it-was.dng [--
from EDITED]
```

`versions` lists earlier contents: path, SHA-256, size, modification time, when the run saw it replaced, and whether a skeleton is held. `restore` writes one content to `--out`, which must not exist. With `--path` alone it takes the path's latest earlier content; `--sha256` names any content. The blocks come from `--from` when

given, otherwise from the first of these that rebuilds it: the file at `--path`, a live file the index lists with that SHA-256, a live file listed with the same block marks (at most 8 are tried). What is written is checked against the SHA-256, and on a mismatch nothing is left behind. `restore` only reads the index, and reads an idle one without writing beside it.

Three limits:

- **The skeleton must exist before the edit.** A file is protected from the run that first indexes it. An edit made before that is not recoverable from the index.
- **A destroyed block is gone.** An edit that replaces a block, such as a regenerated DNG preview or re-encoded image data, cannot be undone from the edited file; `restore` says which block is missing. Keep such contents from another copy.
- **Skeletons are never removed.** The table grows by a few kilobytes per content ever read. For a library of 1.7 million files that is of the order of 15 GB in the index.

Both commands are in the Rust `phindex` only. phindex-ox writes the same three things.

Three states, and never a match on failure

Each identity column is in one of three states:

State	Value column	`_err` column	Example
computed	the value	NULL	a supported DNG's `lr_digest`
failed or refused	NULL	the reason	`dngsem_err` = `read outside the bytes` for a file too short to be a TIFF
not attempted	NULL	NULL	`dng_blocks` for a text file, which has no TIFF header

A NULL value is never a match. SQL already treats it that way, since `NULL = NULL` is not true, and the queries in this manual rely on it: two files that both failed an identity have not agreed on anything. The reason is kept so that you can tell an identity that does not apply from one that broke, and count either.

A file whose identification failed as a whole, because it could not be read or because it changed while it was being read, keeps `sha256` NULL and every identity column NULL. Its reason is printed as an `ERROR` line on stderr (the first 50 of a run are printed) and counted in the run's errors. It is read again on the next run.

Where it fits

This book is one of nine on the FirstPair photo tools. Each owns one part, and The FirstPair Photo Stack shows how they fit together.

- **tiffgraph** walks a TIFF or DNG file's structure under a fixed grammar and finds its editable XMP packet.
- **phidentify** computes a photo's identities: SHA-256, DNG raw blocks, image data, block marks, Lightroom fingerprint, metadata record and preserved digest, and the skeleton that rebuilds a file from an edited copy.
- **phindex** keeps those identities and a skeleton for every file of a volume, aligns two volumes by content, and restores a file as it was before a metadata edit.
- **DNG Compare** compares pairs and preserves one side byte for byte as deltas against the other, restorable even after the other side is edited, with a searchable metadata section, through `dngc`.

- **Semantic Unison** synchronizes photo trees, sending metadata edits without resending image bytes, keeps the versions it replaces as small skeletons, and preserves what a shuttle disk brings into the same archives.
- **packdir** keeps directories of small files as SQLite archives that still work like plain files.
- **Tursolite** is the SQLite and Turso layer under packdir, DNG Compare and Mechanic.
- **diskproxy** serves a folder as a virtual disk, such as previews with camera metadata spliced in, and captures ratings into sidecars.
- **Lightroom Repair** repairs Lightroom Classic catalogs, previews and Sync without forging their state.

What it is not

- **Not a writer of files.** Indexing, aligning and restoring only read the indexed tree and write nothing in it. The commands that change a tree are `conform apply` and `mv --files` (with their `undo`), and they only rename: no file's content is written, replaced or removed.
- **Not a synchronizer or a deduplicator.** It tells you which files agree on which identity. What to copy, keep or delete is your decision, made from queries.
- **Not a catalog tool.** It can compare its `lr_digest` column with a Lightroom Classic catalog's stored fingerprints, read from a copy of the catalog. It never opens a live catalog and never writes one.
- **Not a server.** One indexer, `phindex` or `phindex-ox`, writes an index at a time. Readers can query the index while it runs, because it is in WAL mode.

Building and running

phindex (Rust)

The Rust toolchain is pinned in `rust/rust-toolchain.toml` to 1.99.0; rustup installs it on first use. phindex depends on phidentify from GitHub at a pinned commit, with the `lightroom`, `metadata` and `semantic` features.

```
git clone https://github.com/firstpair/phindex.git
cd phindex/rust
cargo build --release
cargo test --release
```

The result is one binary, `rust/target/release/phindex`. Index a tree:

```
phindex --db archive.sqlite --root /archive --workers 16
```

Run the same command again whenever the tree may have changed. To index a folder you have just added, without walking the rest:

```
phindex --db archive.sqlite --root /archive --only /archive/2026/new-card
```

To compare two indexed trees, use the `align` subcommand, described in its own chapter:

```
phindex align --left old.sqlite --left-root /old --right new.sqlite --right-root /new
--out report
```

phindex-ox (stock OCaml and OxCaml)

phindex-ox is built on phidentify's OCaml package, which binds phidentify's Rust library. It needs OCaml 5.1 or later, stock OCaml or OxCaml, dune 3.9 or later, the `sqlite3` opam package, zlib, and opam 2.1 or later for `--subpath`. Build phidentify's Rust library with the metadata identity first, then pin its OCaml package into the switch you want:

```
git clone https://github.com/firstpair/phidentify
(cd phidentify/rust && cargo build --release -p phidentify-ffi --features metadata)
PHIDENTIFY_FFI_LIBDIR=$PWD/phidentify/rust/target/release \
  opam pin add phidentify ./phidentify --subpath ocaml
cd phindex/ocaml && dune build
./_build/default/phindex_ox.exe --db archive.sqlite --root /archive --workers 16
```

The installed name is `phindex-ox`. The same source builds in a stock OCaml switch and in an OxCaml switch; phidentify's package compiles the domain pool that suits the switch, and the walk line reports which one is running (`domains-stock` under stock OCaml, `domains-oxcaml` under OxCaml). phindex-ox refuses to start when the linked phidentify lacks the `+lightroom` or `+metadata` identities its revision stamp promises.

phindex-ox's C stub reads the nanosecond mtime from macOS's `st_mtimespec`, so as written it builds on macOS. The Rust phindex uses only portable Unix metadata.

The command line

Both indexers take the same options; phindex-ox has one more.

Option	Default	Meaning
`--db FILE`	required	the SQLite index; created if missing, its schema migrated if older
`--root DIR`	required	the tree to index; canonicalized, so stored paths are absolute and free of symlinks
`--only SUBDIR`	none	walk only this folder, which must lie under `--root`; repeatable
`--full-stat`	off	list every directory and stat every file, instead of the quick walk
`--workers N`	8	identification workers: rayon threads in phindex, OCaml domains in phindex-ox; at least 1; see *Choosing – workers*
`--progress-secs S`	60	seconds between progress lines on stderr
`--batch B`	workers × 8	phindex-ox only: files handed to the domain pool at a time

Paths are stored as the canonical absolute path. On macOS, for example, a root given under `/tmp` is stored under `/private/tmp`. Write queries against the stored form.

What it prints

Everything goes to stderr. A run prints a launch header, the walk, the plan, progress lines, and a summary:

```
phindex 2f2c4580ffbf542a1bb4edba4ce1b9327a575ff6 built Mon 2026-11-02 13:50:12 PST
walk: 3104 directories, 12 listed, 3092 unchanged and taken from the index, 1 s
walked 1612934 files under /archive, 412 to read (9.8 GB; 0 of them only for block
marks and a skeleton, from their structures), 37 moved or renamed (identities kept), 5
gone, 0 errors, 16 workers (phindex), identities: phidentify 0.1.0 +lightroom
+metadata; tiffgraph dng-xmp/0,1,2 max_xmp=1MiB; SDE/0-preserved; image-data/1; block-
marks/1
+0h01m 288/412 files, 6.9/9.8 GB (70.4%), 115 MB/s, 0 errors, ETA 0h00m; raw blocks
263, Lightroom 251, metadata 288, preserved 259, image data 21, block marks 284,
skeletons 288, upgraded unread 0
done: read 412 files, 9.8 GB, 0 errors (0 stored SHA-256 mismatches); raw blocks 381,
Lightroom 362, metadata 412, preserved 376, image data 30, block marks 407, skeletons
412; upgraded without a read 0
```

The numbers are illustrative. The launch header is described below. The walk line says how many directories were listed and how many were taken from the index. The plan line says how many files must be read, how many were carried across a move, and how many vanished. The progress line counts, for the files read so far, how many got each identity, and how many rows of the previous revision were settled without a read. Its ETA assumes the remaining bytes go at the rate so far. In phindex the writer wakes every second, so progress lines arrive on time; in phindex-ox they are checked after each batch. In a run that upgrades an

index from the previous revision, the plan line says how many of the files are there only for block marks and a skeleton.

The launch header

The first line names the build: the source commit the binary was built from, with `-dirty` appended when the working tree had uncommitted changes, and the time of the build. phindex-ox adds the domain pool it was compiled with:

```
phindex 2f2c4580ffbf542a1bb4edba4ce1b9327a575ff6 built Mon 2026-11-02 13:50:12 PST
phindex-ox 2f2c4580ffbf542a1bb4edba4ce1b9327a575ff6 built Mon 2026-11-02 13:52:40 PST
(domains-oxcaml)
```

A long indexing run outlives many edits to the source, and the log of a run that took three days should say which code wrote the rows. In phindex, `rust/build.rs` runs `git rev-parse HEAD` and `git diff --quiet HEAD` at build time and passes the result and the build time to the compiler as environment variables, which the binary prints with `env!`; it asks Cargo to rebuild when `.git/HEAD`, `.git/index`, `src` or `Cargo.toml` change, so a commit or an edit refreshes the header. In phindex-ox, a dune rule with `(deps (universe))` runs the same two git commands on every build and writes them into a generated module, `build_info.ml`, which the program prints. A source tree without git prints `unknown` for the commit. The `-dirty` mark says the binary may not match any commit; for runs whose rows matter, build from a committed tree.

The runner script

A first run of a large tree takes days, and it is easy to start one from the wrong build, let the machine sleep under it, lose its output with the terminal, or start a second run on the same index by mistake. `scripts/phindex-run.sh` takes care of those four things and nothing else:

```
scripts/phindex-run.sh --db /Volumes/Backup/index/archive.sqlite \
  --root /Volumes/Archive --log /Volumes/Backup/index/archive.log --workers 8
```

Option	Meaning
`--db FILE`, `--root DIR`	required, as for the indexer
`--ox`	run phindex-ox instead of phindex
`--log FILE`	append the run's output, stdout and stderr together, to this file as well as showing it
anything else	passed to the indexer unchanged: `--only`, `--workers`, `--full-stat`, `--progress-secs`, `--batch`

- **This checkout's build.** The script runs `rust/target/release/phindex` from the checkout it lives in, or with `--ox` `ocaml/_build/default/phindex_ox.exe`. It does not build. If the binary is missing it stops and prints the command that builds it (`cargo build --release` in `rust/`, or `dune build` in `ocaml/`). Rebuild after pulling; the launch header then names the commit.
- **Awake.** Where `caffeinate` exists, as on macOS, the indexer runs under `caffeinate -i`, so the machine does not sleep from idleness while the run lasts. A closed lid or a sleep chosen from the menu still puts it to sleep, and the run waits with it.
- **The log.** With `--log`, output goes through `tee -a`, so the file grows by one section per run, each beginning with its launch header. Without `--log`, the output goes to the terminal only.

- **One writer.** The index path is made absolute, and the script refuses to start, listing the processes, when a `phindex` or `phindex_ox.exe` process is running whose command line has `--db` followed by that same absolute path. Every run the script starts passes its path that way, so two runs through the script never share an index. A run started by hand with a relative path, or from a binary named otherwise, is not recognised; do not mix the two on one index.

Stopping and resuming. Ctrl-C stops the run: the interrupt reaches the indexer, `caffeinate` and `tee` alike, and the script exits. The indexer has nothing to clean up. What it committed stays, at most the last uncommitted results (fewer than 500) are lost, and the run leaves no row in `runs`. To resume, run the same command again. Its quick walk takes the unread files' rows from the index, sees them incomplete and reads them; nothing committed is read again (see *Commits, interruptions and resuming*). With `--log`, the last lines before the interrupt may be missing from the log, since `tee` stops too.

The schema

phindex creates six tables and migrates older ones by adding missing identity columns and tables. Both indexers create the same schema.

files

One row per path ever seen under an indexed root.

Column	Type	Meaning
`path`	TEXT, primary key	canonical absolute path
`size`	INTEGER	size in bytes at the last walk that stat'ed it
`mtime_ns`	INTEGER	modification time in nanoseconds since the epoch
`ino`	INTEGER	inode number
`dev`	INTEGER	device number
`sha256`	TEXT	whole-file SHA-256; NULL until the file is identified
`hashed_unix`	INTEGER	when it was identified, in Unix seconds
`seen_unix`	INTEGER	when the row was created or last reset
`gone`	INTEGER	1 once the path has vanished; the row and its identities are kept
identity columns	TEXT	as in the first chapter, with `ids_rev`

Indexes cover `sha256`, `size`, `dng_blocks`, `lr_digest`, `md_digest`, `preserved`, `image_data` and `block_marks`, so joins on any identity are fast.

A row is **complete** when `sha256` is not NULL and `ids_rev` equals the running indexer's revision stamp. Only complete rows are skipped on a rerun and only complete rows can pass their identities to a moved file.

skeletons

One row per content that has a skeleton, whichever paths held it.

Column	Type	Meaning
`sha256`	TEXT, primary key	the content's whole-file SHA-256
`record`	TEXT	JSON: `sha256`, `size`, `format` (`tiff`, `jpeg` or `whole`), `blocks` (`[at, len, mark, role]` each), `skeleton_bytes`, `skeleton_sha256`, `skeleton_format` (`zstd`)
`skeleton`	BLOB	every byte outside the blocks, as one zstd frame

The table has the columns of DNG Compare's `skeletons.sqlite3`, and the record has that format's members other than the path. Rows are added and never changed or removed.

versions

One row per earlier content of a path.

Column	Type	Meaning
`path`	TEXT	the path
`sha256`	TEXT	the content it held
`size`, `mtime_ns`	INTEGER	that content's size and modification time
`replaced_unix`	INTEGER	when a run first saw the path with other content

The primary key is `(path, sha256, mtime_ns)`. A row is written when a run resets a path's row because its size, mtime or inode changed. A path that vanishes gets no row: there is then no file to take blocks from.

metadata

One row per identified file that yielded a metadata record.

Column	Meaning
`path`	the file's path, as in `files`
`engine`	`exiftool-rs (via phidentify)`
`record_deflate`	the compacted record, raw-deflated

The record is the JSON object exiftool-rs wrote for the file, **compacted**: every string member longer than 1 KiB, such as a base64 thumbnail or a camera-profile table, is replaced by `long-sha256:<hex>:<length>`, the SHA-256 and byte length of the original string. The compacted JSON is then compressed with raw deflate, without a zlib header, as Rust's flate2 `DeflateEncoder` writes it; phindex-ox's C stub writes the same format. `md_digest` is always taken over the full record, never the compact one, so the stored record serves for comparison and display, and the digest for equality.

Because the records are kept, two files' metadata can be compared field by field later without reading either file. To read one back in Python:

```
import json, sqlite3, zlib
db = sqlite3.connect("file:archive.sqlite?mode=ro", uri=True)
(blob,) = db.execute("SELECT record_deflate FROM metadata WHERE path = ?",
                     ("/archive/2024/IMG_0001.dng",)).fetchone()
record = json.loads(zlib.decompress(blob, -15))
print(record["ExifTool:DngCompareEngine"], record["File:FileType"])
```

The record carries the engine's stamp, `ExifTool:DngCompareEngine`. Compare records only between equal stamps.

dirs

One row per directory ever visited: `path`, `ino`, `mtime_ns`, `seen_unix` and `gone`. The inode and mtime are taken before the directory is listed. This table is what makes the quick walk possible.

runs

One row per completed run: `root`, `started_unix`, `finished_unix`, `walked` (files in the walked scope), `hashed` (files identified), `bytes_hashed`, `gone` (files newly marked gone) and `errors`. Both indexers record the walked scope in `root`: the `--only` folder when one was given, otherwise `--root`. A run that was interrupted leaves no row.

Incremental behaviour

What a rerun reads

A file is read when any of these is true:

- its path is new to the index;
- its size, mtime or inode differs from its row;
- its row is not complete: it was never identified, its identification failed, the previous run was interrupted before reaching it, or its `ids_rev` differs from the running revision stamp.

Everything else is skipped without opening the file.

The revision stamp

Each identified row is stamped, in `ids_rev`, with the identity definitions that computed it:

```
phidentify 0.1.0 +lightroom +metadata; tiffgraph dng-xmp/0,1,2 max_xmp=1MiB; SDE/0-
preserved; image-data/1; block-marks/1
```

The stamp names the phidentify identities and features, the tiffgraph grammars and their XMP limit, the preserved-digest definition, the image-data definition and the block-marks definition. The `0.1.0` in it is phidentify's identity version, not phindex's; phindex 0.2.0 added the `; image-data/1` part and 0.4.0 the `; block-marks/1` part. It is the same constant in `phindex` and `phindex-ox`, so either can continue an index the other started.

The stamp changes only when an identity's definition changes: a new grammar, a different limit, a fix that changes some files' values. A release that leaves the definitions alone keeps the stamp, because a new stamp makes every row incomplete and the next run reads every file again. That full re-read is also a check: when it re-reads a file whose size, mtime and inode are unchanged and finds a different SHA-256 from the one stored, it prints `SHA256 CHANGED with unchanged size/mtime/inode`, counts the file as an error and in the summary's stored SHA-256 mismatches, and stores the new values. On an archive, such a line deserves attention: something changed the bytes without changing the metadata the file system keeps.

Upgrading from the previous revision

Adding `block_marks` changed the stamp, and by the rule above that would make every row incomplete and send the next run through the whole archive again: days of reading on a large tree. phindex 0.4.0 avoids that for exactly one step, from the previous stamp (the current one without `; block-marks/1`) to the current one. A row of the previous revision whose file is unchanged keeps all its other identities, and what it lacks comes from the file's structures alone:

- A worker opens the file and builds its skeleton: it reads the header and directories, the bytes outside the blocks, and each block's first and last 4,096 bytes. It never reads a whole block, so a 50 MB raw costs a handful of small reads.
- The row gets `block_marks` or its reason and the new stamp, and the skeleton is stored under the SHA-256 the row already holds. Nothing else in the row changes.

The progress and summary lines count these rows as `upgraded unread`. Files that are new or changed are read in full as always; the upgrade applies only to rows whose size, mtime and inode are unchanged.

Because the stored SHA-256 is trusted for them, the check for a silently changed file, described above, does not run for these rows in the upgrading run.

On a synthetic tree, phindex, phindex-ox under stock OCaml and phindex-ox under OxCaml each upgraded an index another of them had written, and all three ended with the same rows and the same skeletons as a fresh index. On a real archive on hard disks, one folder of 12,365 files (346 GB, mostly DNGs) was upgraded in 149 seconds with 8 workers: 83 files per second, no file read in full, 0 errors, 655 MB of memory at its peak. 12,331 of the files got block marks and a skeleton, 125 MB of skeletons in all, about 10 KB each; the other 34 were videos. At that rate an index of 1.7 million files is upgraded in about six hours.

The upgrade covers only that one previous stamp. A row with any older stamp, such as one written before `image_data`, is read in full.

Skeletons from a store that already has them

The upgrade above opens every file. When a DNG Compare archive already keeps skeletons of the same contents, they can be copied instead, and no file of the tree is opened:

```
phindex import-skeletons --db archive.sqlite --from /Volumes/Backup/dngc/Archive/
skeletons/skeletons.sqlite3
```

For each row at the previous revision whose SHA-256 the store has, this copies the skeleton, computes `block_marks` from its block list, and stamps the row as current. A row says which content it was hashed from, and the skeleton is of that content, so the file is not needed. The result is what an upgrade from the files writes: phidentify cuts files by the archive format's rules and both tools pack skeletons with the same compressor, and a test holds the imported rows equal to the indexer's own, record and bytes.

- **What it leaves.** A row whose content the store does not have stays at the previous revision, and the next ordinary run upgrades it from its file. That is every file the archive never used as a reference, and every file changed since.
- **What it checks.** Each skeleton's bytes are checked against the SHA-256 in its own record, and its blocks for order, before it is copied. One that fails is reported and not imported.
- **What it reads and writes.** The store is opened read-only and immutable. Only the index is written. Another index's `skeletons` table serves as a store too, so one index can be filled from another.

Stop any indexer writing the same index first, as for any second writer.

Rows for copies, from the other tree's index

After a synchronizer has copied files from one indexed tree into another, every copy is new to the receiving index, and an ordinary run would read each in full. The sending tree's index already holds what that read would find. `adopt` takes it from there:

```
phindex adopt --db new.sqlite --root /new/foto --from old.sqlite --from-root /old/foto
```

For each file the other index lists under `--from-root`, `adopt` looks at the same relative path under `--root`. Where this index has no current row and the file there has the size and modification time the other index has for its own file, the row is written from the other index: SHA-256, every identity, the metadata record and the skeleton, with this file's own inode and device. No file of either tree is opened, so half a terabyte of copies is indexed in the time it takes to stat them.

- **The evidence is path, size and modification time.** The copy must have been made with times kept. A volume that keeps whole seconds is allowed for: a time equal to the other one cut to the second counts as equal. This is the trust the incremental run itself places in size and time; `adopt` does not hash anything to confirm it.
- **What does not qualify is left alone.** A file with another size or time, a path with no file, and a file the other index has not read are counted and left for an ordinary run, which reads what it must.
- **A replaced file keeps its earlier content.** If this index knew another content at the path, that content is listed in `versions`, as an ordinary run would list it, and `restore` rebuilds it from the new file.
- **The metadata record is the other file's**, with the folder it names changed to this tree's. Its file-system dates remain those the other index saw.

Run `adopt` after the synchronization and before the next ordinary run. That run then finds the adopted rows current and reads only what `adopt` left.

The quick walk

A directory's mtime changes whenever an entry is added to it, removed from it or renamed in it. phindex uses that. Every directory it visits is stored in `dirs` with its inode and mtime. On a rerun, for each directory:

1. phindex stats the directory.
2. If its inode and mtime equal the stored ones, the directory is **not listed**. Its files come from the index, with the size, mtime and inode stored for them, and its subdirectories come from `dirs`; each subdirectory is stat'ed in turn and goes through the same test.
3. Otherwise the directory is **listed**: every entry is stat'ed, new files and subdirectories are found, and vanished ones drop out.

So a rerun lists only the directories whose contents changed, such as the parent of a new folder and the folder itself, and stats one inode per unchanged directory. On a tree of 1.6 million files in 3,100 directories, the quick walk took 16 seconds. The first run of a tree, before `dirs` knows it, lists everything.

The directory's mtime is taken before it is listed. If an entry is added while the directory is being listed, the stored mtime is older than the directory's, so the next run lists it again.

Symbolic links are not followed, and only directories and regular files are indexed. A name that is not valid UTF-8 cannot be stored as SQLite TEXT; it is skipped and counted as an error.

In-place rewrites and `--full-stat`

The quick walk trusts a directory's mtime to say whether anything in it changed. That holds for every change to the directory's **entries**, but not for a change to a file's **contents**. A file rewritten in place under the same name, by an editor saving into it or a tool appending to it, changes its own mtime and size but not its directory's. The quick walk takes that file's row from the index and never sees the change.

`--full-stat` turns the quick walk off: every directory is listed and every file stat'ed. A file whose size, mtime or inode changed is then read again, exactly as on a first run. Use it after anything that may have rewritten files in place, such as a tool that writes metadata into DNGs, and periodically as a sweep. It costs one stat per file, not one read: on a large tree, minutes rather than seconds, and still nothing like reading the files.

Programs that save by writing a new file and renaming it over the old one change the directory's mtime, and the quick walk finds them.

`--only`

`--only SUBDIR` limits a run to one folder under `--root`, typically one you have just copied in. Only that folder is walked; only rows under it are considered for reading or for being marked gone; files elsewhere are left exactly as they are. The folder must lie strictly under the root, after both are canonicalized.

`--only` may be repeated. Each folder is then indexed in turn, in the order given, as if by separate runs one after the other: each has its own walk, plan, reading and row in `runs`. That is the way to index several new folders in one command:

```
phindex --db archive.sqlite --root /archive --only /archive/2026/card-a --only /
archive/2026/card-b
```

The caution goes the other way. `--only` is a limit for one run, not a setting stored in the index. A later run without `--only` walks the whole root, and if the index has never covered the rest of the root, it lists every directory there and reads every file in it. To keep working on part of a tree, keep passing `--only`, or point `--root` at that part.

Moves are the exception, by design: a file that arrives in the `--only` folder from elsewhere in the index still inherits its identities, and its old path is marked gone even though it lies outside the folder, because the old path demonstrably no longer exists.

Moves and renames

Moving or renaming a file within one file system keeps its inode, size and mtime. phindex uses that to carry identities across a move without reading the file. For each path that is new to the walked scope, it looks for rows anywhere in the index with:

- the same device, inode, size and mtime;
- the same revision stamp, and a SHA-256, so they are complete;
- a path that is not in this walk and no longer exists on disk.

If there is one, the new path takes over its identities and its metadata record, and the old path is marked gone. Renaming a folder of three files is a run that reads nothing and reports three moved and three gone.

The last condition matters for hard links. Two live hard links share an inode, size and mtime, but both paths exist, so neither is taken for a move: each is read and stored separately.

A file moved twice can leave two old rows with the same device and inode, for example when it went from one folder to another between runs, was indexed there with `--only`, and was then moved again. Both old paths have vanished. The new path inherits from one of them, and every vanished old row that is not yet marked gone is marked gone, so the index never shows two live paths for one file that exists once.

A move across file systems is a copy and a delete. The new file has a new inode, so it is read; the old path is marked gone.

Gone

A path that was in the index under the walked scope and is no longer found by the walk is marked `gone = 1`. Its row and its identities stay, because they are evidence: they say what was there. Queries about the current tree add `gone = 0`. A path already marked gone is not counted again. Directories that vanish are marked gone in `dirs` the same way, and a directory that reappears is listed in full.

A gone path can come back. If it comes back unchanged, with the same inode, size and mtime it had when it vanished, as when a file is moved away and then moved back, it is marked present again (`gone = 0`) and keeps its identities; a complete row is not read again. If it comes back as a different file, with a new inode or a new size or mtime, its row is reset and read like any changed file.

Commits, interruptions and resuming

A run has two phases.

The plan is committed in one transaction before any file is read. It inserts rows for new paths and resets rows for changed ones (clearing every identity), marks vanished files and directories gone, records every visited directory's inode and mtime in `dirs`, and applies moves.

The reading commits every 500 results. phindex has one writer: the workers identify files in parallel and stream results over a bounded channel to the main thread, which writes them and commits after every 500 results, successes and failures alike. phindex-ox writes each batch's results in input order and commits after the batch that brings the count to 500 or more.

If a run is stopped, by Ctrl-C, a crash or a power cut, at most the uncommitted results are lost. The plan is already in the index: the files not yet read have rows with a NULL `sha256`, so they are incomplete. The next run's quick walk finds their directories unchanged, takes their rows from the index, sees them incomplete and reads them. Nothing already committed is read again. A run that finishes writes its row in `runs`.

One indexer writes an index at a time. Do not run phindex and phindex-ox, or two copies of either, against the same database at once.

A file that changes during the run

The plan records each file's size, mtime and inode. After phidentify has identified a file, the indexer stats it again. If it is no longer a regular file with the same size, mtime and inode, the result is discarded with the reason `changed since the walk`, counted as an error, and the row stays incomplete for the next run. Inside phidentify, a file that changes while it is being read is likewise an error, never a mixture of old and new identities.

Two implementations, one index

There are two indexers because the same work exercises two runtimes differently, and an index that both can write is a strong test of each. Both write the same tables with the same revision stamp, so either can continue the other's index, and they must agree row for row.

phindex: Rust

`phindex` is one binary. It calls phidentify's Rust core directly, with every identity requested, including the tiffgraph walk through tiffgraph's Rust crate, so the walk and every digest are taken from the same in-memory bytes.

Identification runs on a rayon pool of `--workers` threads. The files to read are sorted by path and handed to the pool one at a time, in that order (rayon's `par_bridge` over the sorted list), so the workers are always reading neighbouring files; the chapter on hard disks explains why that matters. Each worker identifies a file, re-stats it, and sends the result over a bounded channel of 256 results to a single SQLite writer on the main thread. The writer compacts and deflates the metadata record, writes the row, commits every 500 results, and prints progress on time even when no result has arrived. The binary uses mimalloc as its global allocator.

phindex-ox: OCaml, stock and OxCaml

`phindex-ox` is written in OCaml on phidentify's OCaml package, which binds phidentify's Rust library through its C ABI and adds a domain pool and the tiffgraph walk in OCaml.

- **The pool depends on the switch.** Under **OxCaml**, phidentify's package uses a `Domain.Safe.spawn` pool. `Domain.Safe.spawn` accepts only portable closures, so the workers share nothing but an immutable array of paths and an atomic counter; the C externals carry the `@@ portable` modality. Under **stock OCaml 5**, the package uses a `Domain.spawn` pool with the same structure. dune picks the pool for the switch; phindex-ox's own source is the same in both.
- **The tiffgraph walk runs inside the workers.** Each worker opens the file read-only, builds a tiffgraph reader over positional reads on its own descriptor, walks it under the grammars from strictest to loosest, closes it, and passes the packet's position to the Rust identification. Nothing non-portable crosses domains. The C stubs release the OCaml runtime lock while Rust hashes and while the walker reads, so the domains work in parallel under stock OCaml and under OxCaml alike.
- **Work is handed out in batches, in order.** phindex-ox takes `--batch` consecutive files at a time (by default eight per worker) and passes them to the pool, which spawns its domains for that batch; each domain takes the next file from the atomic counter until the batch is done, and the results come back in input order. The writer then writes the batch, commits once 500 or more results are pending, and checks whether a progress line is due. The batch size trades the cost of spawning domains against the time a slow file can hold a batch open.

Rust allocations inside phidentify's C library use mimalloc; the OCaml heap is the OCaml runtime's own.

The comparison

The two indexers were run on the same folder of 1,061 files, 4.6 GB, with 16 workers and a warm file cache, each into its own new index. phindex-ox was built in an OxCaml switch, so it ran the `Domain.Safe.spawn` pool.

- **The results were identical.** Every row of `files` and `dirs` matched in every column, error reasons included. The decompressed metadata records differed in one member only, the file access time, which changes when a file is read and which the metadata digest leaves out.
- **phindex took 13 seconds and 2.3 GB of memory.** phindex-ox took 25 seconds and 3.5 GB.

Equal results are the point: the identities do not depend on which runtime asked for them. For 0.2.0 the check was repeated on real folders with `image_data`: phindex and phindex-ox wrote identical rows when indexing the folders fresh and when upgrading them from the previous revision. For 0.4.0 it was repeated on a synthetic tree of TIFFs, JPEGs, text files, a sidecar and a file with no walker, with phindex and with phindex-ox under stock OCaml and under OxCaml: the three wrote identical `files`, `skeletons` and `versions` rows when indexing fresh, after an edit, and when each upgraded an index another had written. The stored metadata records differ between any two runs, because they hold the file's access time. The comparison measured totals and did not attribute the difference in time and memory. Two structural differences are the obvious candidates: phindex streams results from a pool that never stops, while phindex-ox spawns domains per batch and waits for each batch's slowest file before the next begins. No stock OCaml timing was taken, so the comparison says nothing about stock OCaml's speed relative to OxCaml's.

Files over 1 GiB

phidentify holds a file in memory to identify it only up to 1 GiB. A larger file, typically a video or a large scan, gets a streamed SHA-256 and its metadata record, and the identities that need the bytes in memory report `file over the in-memory limit`. The tiffgraph walk does not need them: it reads only the file's header and IFDs, at known offsets.

The 0.1.0 manual listed this as an open item. phindex-ox's walk had always run on positional reads from its own file descriptor, so for a file over 1 GiB it recorded a profile or a grammar's refusal, while phindex, whose walk ran over the in-memory bytes, recorded `file over the in-memory limit` in `dngsem_err`; on such files the two indexes could disagree. phidentify's Rust core now walks a file over the limit from disk with tiffgraph's `FileReader`, the Rust counterpart of what the OCaml package does, so both indexers record the same `dngsem_profile` or reason. The `preserved` digest and `image_data` still need the bytes in memory and report the limit in both.

A file over the limit still gets block marks and a skeleton when it is a classic TIFF or a JPEG, because the skeleton is built by positional reads and never needs the file in memory. A video over the limit has neither.

Which to use

For production runs, use `phindex`. Use `phindex-ox` when you want the indexer in OCaml, and as a cross-check: run both on a sample, then compare the two databases column by column.

Performance and tuning on hard disks

The first run of a large archive reads every byte, and on an array of hard disks that can take days. Whether more workers make it faster depends on which of two things limits it at the moment: the CPU or the disks.

Where the time goes

For most files, identification costs about one pass over the bytes: a SHA-256, a TIFF parse, a hash of the raw blocks, a metadata extraction, and the preserved digest. Those are fast and the disks usually limit them.

The exception is the Lightroom fingerprint. To compute it, phidentify fully decodes a DNG's raw image, which takes about 0.85 s of CPU per DNG for typical camera files, far more than reading the file. An archive of DNGs is therefore **CPU-bound** by that decode: with N cores busy, it identifies at most about N / 0.85 DNGs per second, whatever the disks could deliver. A folder of JPEGs, sidecars or non-DNG raw files has no decode and is limited by the disks.

When workers help

While the CPU limits, more workers help, up to the number of cores. The sign is that the CPU is busy across all cores, the MB/s on the progress line rises when you add workers, and the disks are not at their limit. A folder of DNGs, where the Lightroom decode dominates, is the usual case.

When the disks limit

Once the disks limit, more workers stop helping and can hurt: every worker reads a whole file, and on hard disks several concurrent readers make the heads move between files. Watch two numbers with a disk monitor such as `iostat`:

- **the read rate**, in MB/s per disk and in total. If it stays flat as you add workers, the disks are the limit.
- **the request size**, the average kilobytes per transfer. Large requests mean long sequential reads, which hard disks do well. If the request size falls as workers rise, the readers are interleaving and the disks are seeking.

The progress line's MB/s is the indexer's own view of the same thing.

When the enclosure limits

Sometimes neither the CPU nor the disks set the limit, but the box the disks are in. A multi-disk enclosure has a bridge between its disks and the cable, and the bridge has a ceiling of its own, which need not be the cable's.

This is a measurement on one enclosure, not a rule: a RAID enclosure connected over Thunderbolt, whose internal bridge is a USB 10 Gb/s one, read at about 240 MB/s, and that figure did not move with more workers or with a larger read size. More workers did not help. Other enclosures will have other ceilings, and a USB 10 Gb/s link is not in itself limited to that rate; the cap belongs to that bridge.

The sign is a read rate that stays flat whatever you change on the computer's side: more workers, fewer workers, larger reads. To confirm it, copy one large file off the enclosure with `cp` or `dd` while nothing else reads from it; if a plain sequential copy reaches about the same rate as the indexer, the indexer already gets everything the enclosure gives. Then use the fewest workers that hold the rate, since on hard disks the

extra workers only add seeks. The system's view of the connection, such as macOS's System Information listing the enclosure under USB rather than Thunderbolt, may show which bridge is inside.

Reading in path order

Which files the workers read at the same moment matters as much as how many workers there are. In 0.1.0 the Rust pool took the list of files with rayon's `into_par_iter`, which splits a list into contiguous ranges and gives each thread its own range. With 16 workers that meant 16 readers in 16 distant parts of the tree, typically 16 different folders years apart, all at once. Each worker's file was contiguous on disk, but the disks served the 16 readers in turn, and every switch between them was a long seek.

On a two-disk hard-disk RAID 0, `iostat` showed what that costs: about 368 transfers per second of about 0.5 MB each, nearly every one a seek, for about 175 to 181 MB/s in total, with the process using about 930% CPU. Now the files are sorted by path and handed out one at a time in that order, so the 16 workers read 16 neighbouring files of the same folder, and the heads move a short way between them. The same array then read at about 212 MB/s, with all workers in one folder, and the process's CPU use fell to about 480%: the run had become disk-bound, which on hard disks is where it should be. phindex-ox already handed out consecutive batches of the sorted list and needed no change.

Choosing `--workers`

A worker does two things with each file, one after the other: it reads the whole file, and then it computes. The computing is the SHA-256 and other digests, the metadata extraction, the tiffgraph walk, and for a supported DNG the full raw decode behind the Lightroom fingerprint, which is by far the most expensive step. A worker waiting on the disk uses no CPU; a worker decoding a DNG uses a whole core. So the right number of workers is the number that keeps whichever of the two is the bottleneck busy, and no more.

Tell the two cases apart from the outside:

- **CPU-bound.** The process's CPU use, in `top` or Activity Monitor, is close to `--workers` × 100%: every worker is computing. The disks show spare capacity in `iostat`. More workers will help, up to the number of cores. This is the usual state in folders of DNGs.
- **Disk-bound.** The process's CPU use is well below `--workers` × 100%: workers are waiting for reads. In `iostat`, the disks' transfers per second (tps) are near what the disks can do, and the kilobytes per transfer (KB/t) tell you whether those transfers are long sequential reads (large KB/t, good) or short ones between seeks (small KB/t, bad). More workers will not help, and on hard disks they can make the seeks worse.

As a starting point on a four-disk hard-disk RAID 5, use 8 workers; such an array was observed at 264 to 350 MB/s with 8. Then watch for a few minutes: if the run is CPU-bound and the disks have room, add workers; if it is disk-bound with small transfers, try fewer and keep the smaller number that gives the same files per second.

When several indexers share a machine, for example one run per volume, keep their workers' total below the number of cores. Each run that becomes CPU-bound wants all its workers on cores, and if the totals exceed the cores, every run slows down, the decode-heavy ones most, and the machine has nothing left for anything else.

Small files lower MB/s, not files per second

An archive is rarely all large DNGs. Lightroom's `.xmp` sidecars, Capture One's `.cop` and `.cof` settings files, thumbnails and other sidecars are small, and many of them sit beside each photo. Each costs an open, a stat,

a read and a metadata extraction, and almost no bytes. In a folder full of them, the MB/s on the progress line drops sharply while the files per second barely change. That is not a slowdown, and adding workers will not bring the MB/s back. Judge progress on such folders by files per second, and expect the ETA, which is computed from bytes, to be optimistic while the indexer is among small files and pessimistic among large ones.

Memory

Each worker holds a whole file in memory while identifying it (files up to 1 GiB; larger ones are hashed by streaming and walked from disk), and the Lightroom decode holds the decoded raw image as well, two bytes per sample. Peak memory therefore grows with the number of workers and the size of the largest files in flight. In the comparison above, 16 workers on that folder peaked at 2.3 GB for phindex and 3.5 GB for phindex-ox.

The other memory question is the allocator. Metadata extraction through exiftool-rs makes many short-lived allocations of mixed sizes. Under the macOS system allocator, that pattern fragmented the heap in long multi-threaded runs and kept about 0.5 MB resident per extracted file, without bound. With mimalloc, memory stays flat. So phidentify's C library uses mimalloc for every Rust allocation, which covers phindex-ox, and the phindex binary sets mimalloc as its own global allocator. If you embed phidentify's Rust crate in another program, choose the allocator there; the crate leaves that choice to the program.

Queries

Open the index with `sqlite3 archive.sqlite`, or read-only from any language. The examples use generic roots; replace them with your stored, canonical paths.

Paths under a root

To select paths under a directory, use a range rather than `LIKE`, as phindex does internally. Every path under `/photos` sorts between `/photos/` and `/photos0`, because `0` is the character after `/`, and the range uses the primary-key index:

```
SELECT count(*), sum(size) FROM files
WHERE path >= '/photos/' AND path < '/photos0' AND gone = 0;
```

Containment between two roots

Which files under one root have no byte-identical copy under another?

```
SELECT a.path FROM files a
WHERE a.path >= '/photos/' AND a.path < '/photos0' AND a.gone = 0
  AND a.sha256 IS NOT NULL
  AND NOT EXISTS (
    SELECT 1 FROM files b
    WHERE b.sha256 = a.sha256 AND b.gone = 0
      AND b.path >= '/backup/' AND b.path < '/backup0');
```

The `a.sha256 IS NOT NULL` line keeps files that were never identified out of the answer; list them separately, since the index does not know yet whether they are backed up. Replace `sha256` with `dng_blocks` to ask the weaker question "is the raw data of every DNG somewhere in the backup, whatever happened to its metadata?"

Metadata-only differences

Which DNGs differ from another file only in their XMP packet? Two files with equal `preserved` digests under the same `dngsem_profile` have every byte outside the packet equal; if their SHA-256s differ, the difference is inside the packet.

```
SELECT a.path, b.path FROM files a JOIN files b
  ON a.preserved = b.preserved AND a.dngsem_profile = b.dngsem_profile
WHERE a.sha256 <> b.sha256 AND a.gone = 0 AND b.gone = 0 AND a.path < b.path;
```

Compare `preserved` only under the same profile: the profile is part of what the digest means. To see what changed in the packet, decompress the two `metadata` records and compare their XMP members, or ask whether `md_digest` differs at all. Two files whose `dng_blocks` agree but whose `preserved` digests differ have the same raw data and some other difference outside the packet, such as a new preview or a moved layout; that is not a metadata-only edit.

Lightroom catalog matching

Lightroom Classic stores a fingerprint of each original in its catalog, in the `digest` column of the `AgLibraryFileDigest` table, with the file size in `fileSize`. `lr_digest` is the same value, in the same uppercase hex, for the DNGs phidentify supports. Attach a **copy** of the catalog, never the live file:

```
ATTACH 'file:catalog-copy.lrcat?mode=ro&immutable=1' AS cat;
SELECT f.path, d.digest FROM files f
JOIN cat.AgLibraryFileDigest d
  ON d.digest = f.lr_digest AND d.fileSize = f.size
WHERE f.gone = 0;
```

Opening the copy with `immutable=1` keeps SQLite from creating journal files beside it; the `file:` form needs URI filenames, which the `sqlite3` shell accepts. An equal fingerprint and size means Lightroom Classic's matcher would accept the file as that original. It does not mean equal bytes or equal pixels. A file with `lr_err` set gets no answer from the index, not a "no": the fingerprint is defined only for the supported strata. Join `AgLibraryFileDigest` to the catalog's other tables to see which catalog entry each digest belongs to.

Copying an index

An index is in WAL mode, which is what lets queries run while the indexer writes. WAL mode is recorded in the database file itself, so a closed index, and every plain copy of one, is still a WAL database, and that changes what a read-only open does with it:

- **A plain read-only open writes beside the file.** Opened with SQLite's `mode=ro` and nothing else, a WAL database needs its `-shm` (shared-memory index) and `-wal` files. Where they are missing, current SQLite creates them beside the database, read-only open or not, and they stay after the connection closes. A backup volume that should hold one file per index then holds three, and a volume mounted read-only cannot take them at all, so the open fails.
- **macOS's own SQLite refuses instead.** The `sqlite3` that ships with macOS fails to open a WAL database read-only when its `-shm` file is missing, with `unable to open database file`, even in a writable folder.

Both were checked on a synthetic WAL database: SQLite 3.54 created `-wal` and `-shm` on a `mode=ro` open, macOS's `sqlite3` 3.43.2 refused it, and both failed in a read-only folder.

`phindex align` opens its indexes with such a plain read-only open, through the SQLite built into phindex. On a WAL-mode index it creates `-wal` and `-shm` beside the file, and in a read-only folder it fails with `attempt to write a readonly database`; both were seen on synthetic indexes. Give it indexes in a writable folder, or copies switched out of WAL mode as below.

There are two ways around it.

Read a closed index with `immutable=1`. `immutable=1` tells SQLite that the file cannot change, so it takes no locks and needs no `-shm` or `-wal`. Use it only while no indexer is writing that index: an immutable reader does not see a writer's changes and can read a half-updated file. DNG Compare's `dngc` opens phindex indexes this way.

```
sqlite3 'file:/Volumes/Backup/index/archive.sqlite?immutable=1' \
  'SELECT count(*) FROM files'
```

Copy it, then take the copy out of WAL mode. Copy the closed index with `cp` to a writable place, switch the copy's journal mode, and then move it where it should live:

```
cp ~/indexes/archive.sqlite /tmp/archive.sqlite                # no indexer running
sqlite3 /tmp/archive.sqlite 'PRAGMA journal_mode=DELETE'
rm -f /tmp/archive.sqlite-shm /tmp/archive.sqlite-wal          # if any were left
mv /tmp/archive.sqlite /Volumes/Backup/index/archive.sqlite
```

The copy is then an ordinary rollback-journal database: a `mode=ro` open reads it without creating anything, also on a read-only volume. The `rm` is there because macOS's `sqlite3` leaves a `-shm` file behind after the switch; a database out of WAL mode does not use it, and once no connection is open it can go. Copy only an index no indexer is writing; a copy taken during a run can miss what is still in the `-wal` file. To continue indexing into such a copy, just run the indexer on it: it switches the file back to WAL mode when it opens it.

Keep a short README beside index copies on a volume, so that whoever finds them later does not open them the way that writes beside them:

```
These are phindex indexes (SQLite), copied here with journal_mode=DELETE.
Read them with a read-only open (mode=ro) or with immutable=1.
A copy still in WAL mode must be read with immutable=1, or copied and
switched with PRAGMA journal_mode=DELETE first; a plain read-only open
creates -wal and -shm files beside it, and macOS's sqlite3 refuses it.
```

Housekeeping

```
-- why identities failed, most common first
SELECT lr_err, count(*) FROM files WHERE lr_err IS NOT NULL AND gone = 0
GROUP BY lr_err ORDER BY 2 DESC;

-- files still to read (never identified, failed, or from an older revision)
SELECT count(*) FROM files WHERE gone = 0
  AND (sha256 IS NULL OR ids_rev IS NOT
       'phidentify 0.1.0 +lightroom +metadata; tiffgraph dng-xmp/0,1,2 max_xmp=1MiB;
SDE/0-preserved; image-data/1');

-- the last runs
SELECT root, datetime(started_unix, 'unixepoch') AS started,
       finished_unix - started_unix AS secs,
       walked, hashed, bytes_hashed, gone, errors
FROM runs ORDER BY started_unix DESC LIMIT 10;
```

Aligning two trees

The containment query in the previous chapter answers one question: is every file's exact bytes somewhere else? Real archives need a finer answer. A shoot copied to a new disk may have been renamed by date, sorted into other folders, had its capture times shifted, had its JPEGs re-rendered and its sidecars rewritten. A byte comparison calls all of that missing. `phindex align` instead accounts for every file under a **left** root in a **right** root by identity, file by file and photo by photo, and says for each what kind of match it found.

```
phindex align --left old.sqlite --left-root /old/foto \
  --right new.sqlite --right-root /new/foto --out report
```

The two roots must be indexed first; they may be in two index files or in the same one. `align` opens the indexes read-only and never reads a photo. It considers the rows under each root that are not gone, and leaves out, on both sides, every file whose name begins with a dot, such as `.DS_Store`, AppleDouble `._` files and the hidden files editors and copying tools leave behind. Only the file's own name counts: a file inside a folder whose name begins with a dot is aligned like any other. A left-out file appears nowhere in the report, not even as unmatched, so a tool that walks the tree itself finds files `align` did not account for; `dngc`, below, keeps them as full copies. On a real trial, a left tree of 1.7 million files was aligned with a right tree of 1.6 million in 24 seconds, using 1.6 GB of memory.

Units

A photo is rarely one file. A raw usually has a JPEG beside it, made by the camera, and an XMP sidecar written by an editor. `align` groups files into **units**: the files of one folder whose names are equal up to the first dot. `IMG_0001.RW2`, `IMG_0001.JPG` and `IMG_0001.xmp` in one folder are one unit, `IMG_0001`; the same names in another folder are another unit.

Each file has a kind from its extension: `dng`; `raw` for the other camera raws (NEF, CR2, CR3, ARW, RW2, RAF, ORF, PEF, 3FR, IIQ and others); `jpeg`; `image` for TIFF, HEIC, PNG and PSD; `sidecar` for XMP sidecars; and `other` for everything else. A unit's **kind** names its images in a fixed order, joined by `+`: `dng`, `dng+jpeg`, `raw`, `raw+jpeg`, `jpeg`, `jpeg+image` and so on. Sidecars ride along and do not appear in the kind; a unit with no image at all is `sidecar` or `other`. The kind separates cases that a count of files would mix up: a `dng` unit is a DNG alone, a `dng+jpeg` unit a DNG with its JPEG, and the report counts them apart.

How a unit is matched

Each left file is matched by the **strongest identity it shares** with some right file, in this order:

Tier	Identity shared	Meaning
`identical`	`sha256`	the same bytes
`xmp-only`	`dng_blocks`, and `preserved` under the same `dngsem_profile`	a DNG whose copy differs only inside its XMP packet
`dng-metadata`	`dng_blocks`	a DNG with the same raw data and some other difference, such as new previews
`image-metadata`	`image_data`	a raw, TIFF or JPEG with the same image bytes and edited metadata

Tier	Identity shared	Meaning
`lightroom`	`lr_digest`	only Lightroom's original fingerprint is shared
`unit-partner`	none	the file's counterpart, by kind, in the right unit its photo matched

A unit is matched as a whole. Its members are taken in order of kind: the DNG first, then other raws, then JPEGs and other images, then sidecars and the rest. The first member that matches anything picks the unit's **target**, the right unit its photo is in. Among several candidates of a tier, a member prefers the same relative path, then a candidate in the target unit, then the same file name, and then the first path in sort order. That is how the members follow their raw: when a camera JPEG has a byte-identical copy in some other folder, the copy in the raw's unit wins, because it sits in the target.

A member that shares no identity at all, but whose unit has a target, takes the target's member of the same kind, if no member matched by identity has taken it, as `unit-partner`. A JPEG re-rendered from its raw, with different image bytes, is the usual case: nothing in it is shared, but it is plainly the same photo's JPEG. A member not yet indexed never becomes a unit partner.

Statuses and flags

Each left unit gets one status:

Status	When
`identical`	every member matched with `identical`
`metadata-differs`	every member matched, some with a weaker tier, but no image is a unit partner
`image-differs`	every member matched, and at least one image member (DNG, raw, JPEG or other image) only as a unit partner
`partial`	some members matched and some did not
`left-only`	no member matched, and every right row is indexed
`not-found-yet`	no member matched, but some right rows are not yet indexed, so a match may still appear
`not-indexed`	at least one member has not been indexed on the left

`image-differs` is reserved for images. A sidecar or settings file that matched only as a unit partner means its text changed, which is a metadata difference, and leaves the unit `metadata-differs`. An unread row is never reported as missing: on the left it makes the unit `not-indexed`, and on the right it turns `left-only` into `not-found-yet` until the right index is complete.

Flags describe how the match was found, and a unit can have several:

- `moved`: the target unit is in another folder.
- `renamed`: the target unit has another name.
- `split`: some members matched files outside the target unit.
- `duplicates`: some member had more than one candidate in its tier.
- `right-has-more`: the target unit has more members than the left unit, such as an added sidecar.

Right-only units

A right unit to which no left file was matched is listed as right-only, with its content: `new` when it holds something the left lacks; `copy` when every member's SHA-256 also occurs on the left, as for a second copy of a left file in another folder; or `not-indexed` while any member is unread. New units are counted by kind.

The report

`--out` names a directory, created if needed, which receives:

File	Contents
`summary.json`	file and unit counts per side, left units by kind and status, flag counts, left files by tier, right-only units by kind, copies, and unread counts; also printed to stdout
`units.tsv`	one line per left unit: unit, kind, status, flags, right unit, and each member's extension with its tier
`files.tsv`	a header line, then one line per left file: left path, right path, tier, and the number of candidates; relative to the roots. A file with no match has an empty right path and the tier `unmatched`, or `not-indexed` when its row is unread
`right-only.tsv`	one line per right-only unit: unit, kind, content
`dng-pairs.tsv`	absolute `LEFT<TAB>RIGHT` paths of DNGs matched by a shared identity whose bytes differ
`image-pairs.tsv`	the same for matched raws, JPEGs and other images
`partner-pairs.tsv`	`KIND<TAB>LEFT<TAB>RIGHT` for image files matched only as unit partners: no shared identity, so not for a pixel comparison

The two pairs files hold only pairs that share an identity. An image file matched only as a unit partner, with nothing in common with its right file but its place in the photo, goes to `partner-pairs.tsv` instead, with its kind (`dng`, `raw`, `jpeg` or `image`) first. A pixel or raw comparison of such a pair can only confirm that the two differ, or fail outright when the right file is damaged, as with a black, undecodable copy saved under a suffixed name beside a good original. These pairs are for review: open them side by side, or decide from `units.tsv` what the unit is. Their three columns also keep them from being passed by mistake where `LEFT<TAB>RIGHT` lines are expected. Identical files, sidecars and other files that are not images appear in none of the three.

Progress

`align` reads both indexes into memory, indexes the right side's identities, matches and writes the report. On archives of more than a million files that takes a minute or two, and so `align` reports on stderr as it goes. Each stage is named when it starts, with its counts, and ends with the seconds since it began; within a long stage, a line comes every 5 seconds:

```
align: reading old.sqlite under /old/foto ...
  412,000 files read (5s)
  1,700,212 files in 812,409 units (21s)
align: reading new.sqlite under /new/foto ...
  1,612,934 files in 790,118 units (19s)
align: indexing the right side's identities ...
```

```
align: matching 812,409 left units ...
  matched (3s)
align: writing report ...
  written (2s)
```

The numbers are illustrative; the 5-second lines appear only in a stage that lasts that long. Before these lines, `align` prints the launch header described under *Building and running*. Standard output carries only the JSON summary, the same as `summary.json`, so a caller can read it from stdout while the progress goes to the terminal or a log.

From alignment to raw comparison

An alignment says which files correspond and how; it does not look at pixels. For DNGs, dng-compare does: it compares raw samples and metadata of file pairs. `dng-pairs.tsv` is written in the format of its `--pairs` option, with absolute paths, so the pairs `align` found can be fed to it directly. With `--pairs-only`, dng-compare compares just those pairs and does not walk or name-match the roots:

```
uv run python compare_dng.py /old/foto /new/foto \
  --pairs report/dng-pairs.tsv --pairs-only --output reports/aligned-dngs
```

Identical files are left out of the pairs files, since there is nothing to compare, and so are sidecars, whose differences the metadata records already show. To compare and also preserve a whole tree in one command, use `dngc`, as the next chapter describes.

A worked example

The integration test for `align`, in `rust/tests/align.rs`, builds two small synthetic trees of minimal RW2 and JPEG files and indexes both. The left folder `s1` holds five raw+JPEG units, `IMG_1` to `IMG_5`, a lone `IMG_6.JPG`, and an `IMG_7.JPG` whose row has not been read. On the right:

- `IMG_1`'s two files moved, unchanged, to the folder `t1`;
- `IMG_2`'s two files had their capture time shifted, which changes their metadata bytes but not their image data;
- `IMG_3`'s raw stayed and its JPEG exists only in another folder, `other`;
- `IMG_4` was renamed to `X_4`, unchanged, and a copy of its JPEG also sits in `dup`;
- `IMG_5`'s raw is unchanged and its JPEG was re-rendered with different image data;
- `IMG_6` is not there, and a new `IMG_9.JPG` is.

`units.tsv` then reads:

Left unit	Kind	Status	Flags	Right unit
s1/IMG_1	raw+jpeg	identical	moved	t1/IMG_1
s1/IMG_2	raw+jpeg	metadata-differs		s1/IMG_2
s1/IMG_3	raw+jpeg	identical	split	s1/IMG_3
s1/IMG_4	raw+jpeg	identical	renamed,duplicates	s1/X_4
s1/IMG_5	raw+jpeg	image-differs		s1/IMG_5
s1/IMG_6	jpeg	left-only		
s1/IMG_7	jpeg	not-indexed		

Both members of `IMG_2` matched as `image-metadata`. `IMG_4`′s JPEG had two identical candidates, the one beside `X_4.RW2` and the copy in `dup`; it chose the one in the target unit, and the unit is flagged `duplicates`. `IMG_5`′s JPEG shares no identity and matched as the unit partner of the raw, so the unit's image differs. `image-pairs.tsv` lists `IMG_2`′s two files for a closer look; `IMG_5`′s JPEG, which shares no identity, is in `partner-pairs.tsv` instead. `right-only.tsv` lists `s1/IMG_9` as `new` and `dup/IMG_4` as a `copy` of left content; `other/IMG_3` is not right-only, since `IMG_3`′s JPEG matched there.

Renaming a tree to match another

An alignment often shows that two trees hold the same pictures under different names: a shoot was moved to another folder in one of them, or its files were renamed. `phindex conform` gives the right tree's aligned files the names they have on the left, by renaming them in place. It never copies or changes a file, so a tree of a million files is conformed without reading one.

It works in three steps, each its own command:

```
phindex conform plan  --report report --db new.sqlite --out conform [--limit N]
phindex conform apply --plan conform/plan.tsv --db new.sqlite
phindex conform undo  --journal conform/plan.tsv.journal --db new.sqlite
```

`--report` is a report of `phindex align` whose right side is the tree to rename, and `--db` is that tree's index. Index both trees and align them just before planning, and run `apply` and `undo` only while no indexer writes the index.

The plan

`plan` reads the report and the right index and touches nothing else. For each left unit whose right unit sits in another folder or has another name, it lists the renames that give the right unit the left unit's folder and name:

- A matched right file takes the path of the left file it was matched to, whatever the tier. A file whose metadata differs is renamed like an identical one, since a rename changes no content.
- A right member that the left unit lacks, such as a sidecar, follows its unit and keeps its own suffix.
- A unit whose folder and name agree but whose files differ in a suffix, say `.jpg` against `.JPG`, is listed with the cause `name`.

`plan.tsv` starts with a line naming the root, then has one line per file: the unit, the path now, the path to be, the size, modification time and SHA-256 from the index, and the cause (`moved`, `renamed` or `name`). Paths are relative to the root. `plan` prints a JSON summary on standard output and does not overwrite an existing plan.

For a trial on a few units, `--limit N` stops the plan before the unit that would take it past N files. Units stay whole, so the plan may hold fewer than N files. The units after that point are neither planned nor listed in `skipped.tsv`, and the summary says the plan stopped at the limit. Apply the trial plan, check the result, undo it or keep it, and plan again without the limit; renames already made are then simply in place.

Units that are left alone

A unit is renamed only when the rename is plainly safe. Otherwise it is listed in `skipped.tsv` with a reason, and none of its files is in the plan:

Reason	Meaning
`split`	the unit's members were matched in more than one right unit
`duplicates`	a member has more than one candidate on the right
`target-shared`	several left units correspond to this right unit, or one of its files is matched to another unit

Reason	Meaning
`not-indexed`	a file of the unit has not been read
`not-in-index`	the index no longer lists a file of the report: index and align again
`destination-taken`	a destination is a live path of the right tree, or another rename of the plan claims it

Names are compared without regard to letter case for `destination-taken`, so a plan made on one volume is safe on a case-insensitive one. A destination held by a file that is itself renamed away becomes free afterwards: index, align and plan again to pick such chains up. Two units that would swap names are never planned.

Applying it

`apply` renames with the system's rename call and never copies. A file keeps its bytes, inode and times; only its name and folder change. A unit is renamed whole or not at all: before any rename of a unit, every file of it must still have the size and modification time of the plan, every destination must be free, and the index must list the files as the plan does. A unit that fails a check is reported, left as it was and counted; `apply` then exits with an error after finishing the others.

Each rename is written to the journal, and the journal to disk, before the rename is made. The journal is `PLAN.journal` beside the plan unless `--journal` names another file. Folders that do not exist are created and noted in the journal. Folders left empty are not removed, unless `--prune-empty` is given: then `apply` removes the folders its renames emptied, and their parents while those are empty too, and notes each in the journal.

The index follows: each renamed file's row, with its metadata record and its earlier versions, is moved to the new path with every identity, and a row marking an older, vanished file at that path is dropped. The next indexing run therefore reads nothing again, and reports nothing as moved.

`apply` may be run again after an interruption. A rename already made is recognised by the file at its destination and counted as such.

Undoing it

`undo` replays the journal from its end with the same checks: a file is renamed back only if it still has the size and modification time recorded, and its old path is free. The index rows move back with it, and a folder that `apply` created is removed if it is empty. `undo` may be run again; renames already taken back are counted as such.

Since a rename changes no content, there is nothing for a skeleton to restore here. Skeletons matter when a file is replaced, which `conform` never does.

What it does not do

- It does not bring files over. A left unit with nothing on the right stays missing; a synchronizer or a copy does that.
- It does not replace a right file whose content differs from the left one.
- It does not remove right-only files, duplicates or emptied folders.
- It does not follow a rename into other records. An archive or catalog that names the right tree's files by path has to find them again by content; a DNGC archive does so through the index.

Setting aside what another tree lacks

`conform` gives aligned files the same names in two trees. What is left is the files one tree has that the other does not: copies kept under a second name, sidecars the other side never had, pictures that were never brought over. `phindex diff` lists them, and `phindex mv` moves a listed set to another folder with the index following, so that what stays is a subset of the other tree.

```
phindex diff --left new.sqlite --left-root /new/foto --right old.sqlite --right-root /
old/foto --out diff
phindex mv --db new.sqlite --files --list diff/diff.tsv --to /new/review
phindex mv undo --journal diff/diff.tsv.journal --db new.sqlite
```

What `diff` lists

`diff` reads the two indexes and opens no file. It gives every file under the left root one state:

State	The right tree has	Aligned
`same`	the same relative path and the same bytes	yes
`same-image`	the same path, other bytes, the same raw blocks or image data	yes
`changed`	the same path, other bytes, no shared image identity	
`unread`	the same path, but one of the two rows is not read yet	
`copy`	no such path; the same bytes somewhere else	
`variant`	no such path; the same raw blocks or image data somewhere else	
`only-left`	no such path, and nothing with its content	
`unknown`	no such path, and the left row is not read yet	

`diff.tsv` lists the files in the states named with `--states`. The default is the four states without the path on the right, `copy,variant,only-left,unknown`: once those files are moved away, the left tree is a subset of the right one by path. A sidecar edited on one side is `changed`, and so is a re-rendered JPEG; they keep their place unless `--states` names `changed` too.

The list starts with a line naming the left root, then a header, then one line per file: its path relative to the root, state, size, modification time and SHA-256. The summary, with files and bytes for every state, goes to `summary.json` and to standard output. Read the list before moving anything: the state column says whether a file's content is safe in the other tree (`copy`), held there with other metadata (`variant`), or held nowhere else (`only-left`). `diff` does not overwrite an existing list.

`align` and `diff` answer different questions. `align` asks where each left unit is in the right tree, by identity, wherever it sits. `diff` asks which left files are at a path the right tree does not have. Run `conform` first, so that files which only differ in name are renamed rather than set aside.

Moving a list

`phindex mv --list LIST --to DIR` moves every file of the list to the same relative path under `DIR`. The option `--files` decides what is moved:

- **Without `--files`, only the index rows move.** That is for files already moved by other means. Each file must be at its new path with the size and modification time the index has; one that is not is refused and left as it was. `--force` moves its row anyway.
- **With `--files`, the files are moved on disk too**, by renaming and never by copying, so `DIR` must be on the same volume. A file already at its new path is recognised and only its row moves.

Each file is checked first, as in `conform apply`: it must still have the size and modification time of the list, its destination must be free, and the index must list it as the list does. A file that fails a check is reported and left; the command finishes the others and then exits with an error. With `--files`, each rename is written to the journal (`LIST.journal` unless `--journal` names another file) before it is made, and `phindex mv undo` replays the journal backwards. The command may be run again after an interruption.

Two more options: `--limit N` takes only the first N files of the list, for a trial, and `--prune-empty` afterwards removes the folders the moves left empty, and their parents while those are empty too. Nothing but an empty folder is ever removed, each removal is noted in the journal, and `undo` creates the folders again.

The index rows move with their identities, metadata records and earlier versions, so an indexing run over either folder afterwards reads nothing again.

Moving one file or one folder

```
phindex mv --db new.sqlite [--files] [--force] SRC DST
```

This moves one file, or a folder with everything under it, with the same meaning of `--files` and `--force`. `DST` is the whole new path, not a folder to move into, and it must not exist: a folder is moved by one rename and is never merged into another. Without `--files`, every file under the folder must already be at its new path, or nothing is changed. There is no journal for this form; the same command with the paths exchanged takes it back.

Feeding dngc

DNG Compare keeps one tree, the left, as a byte-for-byte archive of deltas against another, the right, with a searchable metadata section. Its unified command, `dngc`, can take both what to pair and the metadata from phindex indexes, so that a tree of a million files is archived without reading most of it. This chapter covers only what `dngc` takes from phindex; the archive itself, its layout, verification and restore are in DNG Compare's manual.

```
dngc run /Volumes/Archive/2026 /Volumes/Backup/2026 /Volumes/Backup/dngc-2026 \
  --index archive.sqlite backup.sqlite --metadata phindex
```

The first two arguments are the left and right roots, the third the archive, which lies outside both, and `--index` names the phindex indexes of the left and the right tree, in that order. Both trees should be indexed by a run that has finished.

- **Pairs from `align`.** Without `--pairs`, `dngc run` covers the whole left tree. Given `--index`, it runs `phindex align` on the two indexes into a temporary report and takes its pairs from `files.tsv`: every left file with the right file it matched, at any tier, unit partners included. A left file with no match is kept as a full copy. `dngc` finds `phindex` in the `PHINDEX` environment variable, then in a `phindex` checkout beside its own (`phindex/rust/target/release/phindex`), then on the `PATH`.
- **The left tree must be fully indexed.** If `align` reports any left file as `not-indexed`, or if a walk of the left tree finds a file that is in no line of `files.tsv` and whose name does not begin with a dot, `dngc` stops and asks you to index the left tree first. An unread right row is not an error, but a left file whose counterpart is unread has no match, so it is kept as a full copy; index the right tree completely too.
- **Dot names are kept, not dropped.** `align` leaves out names beginning with a dot. They may still be unique (an editor's hidden file is still someone's file), so `dngc` lists them from its own walk and keeps them as full copies. Its walk skips only `.DS_Store` and symbolic links.
- **A row counts only while it is current.** Wherever `dngc` uses an index row, it uses it only when the path is in `files`, not gone, with a SHA-256, and the row's size and mtime equal the file's as it stats it now. Anything else is treated as having no row: the file is read.
- **Identical pairs are settled without reading.** When a pair's left and right rows are current and have the same SHA-256, the comparison is settled as equal and the preservation needs no delta. Such pairs are recorded first, in bulk, as reference records (the left file is the right file's bytes), written 5,000 to a transaction, before the worker pool takes the rest. No byte of either file is read.
- **`--metadata phindex`.** Without it, `dngc` compares metadata by running Perl ExifTool on the files. With it, no ExifTool runs: the archive's metadata section is built from the metadata records phindex stored for both sides of each pair. Those are exiftool-rs records, stamped `ExifTool:DngCompareEngine`, and a string value over 1 KiB, such as an embedded thumbnail or a camera-profile table, is there as `long-sha256:<hex>:<length>`, so the section shows that such a value differs but not its content; the bytes themselves are in the preserved files. A pair whose rows are not current, or that has no record, is written with the reason and not compared. An archive whose metadata came from Perl ExifTool is refused, since records of different engines are never mixed. `--metadata phindex` requires `--index`.

`dngc` reads the indexes itself with `immutable=1`, and the `align` it runs opens them with a plain read-only open (see *Copying an index*). Run it only while no indexer writes either index, and with the indexes in a writable folder or switched out of WAL mode. With `--pairs FILE`, `dngc` takes `LEFT<TAB>RIGHT` lines from the file instead of running `align`; `dng-pairs.tsv` and `image-pairs.tsv` are such files, and `partner-pairs.tsv`, with its kind column, is not.

Testing

The integration tests in `rust/tests/incremental.rs` run the phindex binary on disposable synthetic trees in a temporary directory, with two workers:

- **Reruns read only what changed.** A first run reads six files; an unchanged rerun reads none; a new nested folder is found by the quick walk and its one file read; a file rewritten in place is not read by the quick walk and is read by `--full-stat`, with a new SHA-256; a deleted file is marked gone and not counted again on the next run.
- **Moves keep identities, and `--only` limits the walk.** Renaming a folder of three files reads nothing and reports three moved and three gone, with the SHA-256 carried to the new path. With `--only` on a new folder, the new file is read and a file moved into the folder from outside it is carried, and its old path, outside `--only`, is marked gone. A file moved away and then back to its old path, unchanged, is not read and is no longer marked gone.
- **`--only` may be repeated.** Two new folders given as two `--only` options are both indexed in one run.
- **Identities are recorded separately.** For a text file, the SHA-256 is present, the DNG identities are absent, the metadata digest is present, the refused grammar leaves a reason in `dngsem_err`, and one metadata record is stored.

The tests in `rust/tests/skeletons.rs` build small TIFFs, JPEGs, a sidecar and a file with no walker:

- **Block marks and a skeleton per content.** A DNG-like TIFF and a JPEG get block marks; a sidecar and a large file of another format get the reason. Two equal files share one skeleton, the sidecar is stored whole, and the large file has none.
- **An edited file is restored as it was.** A file is rewritten twice with a longer XMP packet. Its block marks stay the same, both earlier contents are listed by `versions`, and `restore` writes each back byte for byte: the latest by path, an older one by SHA-256, one found by block marks after the edited file moved, and one from a file the index never saw. An existing `--out` is refused.
- **A content whose blocks are gone is not restored.** After the image data itself is replaced, `restore` fails, names the missing block and leaves nothing behind.
- **The upgrade reads no file in full.** Rows set back to the previous stamp, with the skeletons deleted, come back with the same block marks, reasons, stamp and skeletons, and the run reads nothing in full. A row of an older stamp is read again.

The tests in `rust/tests/conform.rs` align two trees and conform the right one. The plan holds the moved unit with its extra sidecar, the renamed unit whose raw was edited, the suffix that differs only in letter case and the unit that belongs in a folder not yet there, and skips the split unit, the one with a duplicate, the one whose name is taken and the two that share a target. After `apply` the files have the same inodes and index rows under their new paths, an indexing run reads nothing and finds nothing moved, and a second alignment shows the units under their left names. A second `apply` and a second `undo` change nothing, and `undo` restores every name and removes the folders it had created. A unit with a file edited after the plan, or with its destination taken since, is left whole.

The test in `rust/tests/adopt.rs` copies files from one indexed tree into another with their times kept: a replacement, two new files and one copy whose time was not kept. `adopt` writes three rows equal to the other index's in every identity, with this tree's inodes and folder, copies their skeletons, lists the replaced content as a version that `restore` rebuilds, and leaves the fourth file; the next ordinary run reads that one file only.

The tests in `rust/tests/mv.rs` cover `diff` and `mv`. Two trees share one file, differ in the metadata of a second and in a sidecar, and the left one has a copy, a variant and two files of its own at paths the right one lacks. `diff` gives each file its state and lists the four. Without `--files` the list is refused, since no file has been moved. With `--files`, a trial of two and then the rest move the files, the emptied folders are removed, the index rows are at the new paths, indexing either folder reads nothing, and a second `diff` lists nothing; `undo` puts everything back. A second test moves one file and one folder: rows only after the file was moved by hand, rows only by `--force`, the file too with `--files`, a folder's subtree by one rename, and a refusal to merge into an existing folder.

The test in `rust/tests/align.rs` is the worked example of the previous chapter: it checks every unit's kind, status, flags and target, the tiers of `IMG_2`, the two image pairs, the one partner pair and the right-only units.

```
cd rust && cargo test --release
```

The equivalence of phindex and phindex-ox rests on comparing their databases on the same tree, as in the comparison above. Synthetic tests check the indexing logic; they do not survey real camera files, and they do not qualify the identities, which phidentify's manual covers.

Licences

The phindex source is MIT. The binaries link third-party code under other licences, through phidentify:

- **rawler**, under LGPL-2.1, decodes raw samples for the Lightroom fingerprint.
- **exiftool-rs**, under GPL-3.0-or-later, extracts the metadata record, through dng-compare's FFI crate.

Built phindex and phindex-ox binaries are therefore covered by the GPL. phindex reads the indexed tree and writes its SQLite file; only `conform` and `mv --files` change the tree, by renaming.

Reference

Item	Value
Version	0.7.0
Revision stamp	`phidentify 0.1.0 +lightroom +metadata; tiffgraph dng-xmp/0,1,2 max_xmp=1MiB; SDE/0-preserved; image-data/1`
Upgraded without a full read	rows of the stamp without `; image-data/1`
Metadata engine label	`exiftool-rs (via phidentify)`
Tables	`files`, `metadata`, `dirs`, `runs`
Commit interval	every 500 results; the plan in one transaction before reading
Journal mode	WAL
Defaults	`--workers 8`, `--progress-secs 60`, phindex-ox `--batch` = workers × 8
Errors printed	the first 50 per run
In-memory limit	1 GiB (phidentify); larger files get a streamed SHA-256, the metadata record and the tiffgraph walk from disk
`image_err` reasons	`DNG: the raw-block identity applies instead`; `not a TIFF-family file or JPEG`
Launch header	`phindex <commit>[-dirty] built <time>`; phindex-ox adds its domain pool
Long strings in records	over 1 KiB, as `long-sha256:<hex>:<length>`
Record compression	raw deflate, no zlib header
Rust	1.99.0, edition 2024; rayon pool fed in path order, one SQLite writer, mimalloc
`align`	units by folder and name up to the first dot; tiers `identical`, `xmp-only`, `dng-metadata`, `image-metadata`, `lightroom`, `unit-partner`
`align` report	`summary.json` (also on stdout), `units.tsv`, `files.tsv`, `right-only.tsv`, `dng-pairs.tsv`, `image-pairs.tsv`, `partner-pairs.tsv`
`align` progress	stderr: each stage with counts and seconds, a line every 5 s within a stage; names beginning with a dot left out
`conform`	`plan` (report and right index to `plan.tsv`, `skipped.tsv`; `--limit N` files for a trial), `apply` (renames in place, journal `PLAN.journal`, index rows follow), `undo`; whole units only; skips `split`, `duplicates`, `target-shared`, `not-indexed`, `not-in-index`, `destination-taken`
`adopt`	rows for copies from another index: same relative path, size and modification time (whole seconds allowed); no file opened
`diff`	left files by state: `same`, `same-image`, `changed`, `unread`, `copy`, `variant`, `only-left`, `unknown`; `diff.tsv` lists `--states` (default: the last four), `summary.json`
`mv`	`SRC DST` (a file or a folder's subtree) or `--list DIFF.tsv --to DIR`; index rows only, or the files too with `--files`;

Item	Value
	`--force`, `--limit N`, `--prune-empty`, `--journal FILE`; `mv undo`
Runner	`scripts/phindex-run.sh [--ox] [--log FILE] --db INDEX --root DIR [options]`: this checkout's build, `caffeinate -i`, appended log, one writer per index
Index copies	read closed indexes with `immutable=1`, or copy and set `PRAGMA journal_mode=DELETE` on the copy
OCaml	5.1 or later: stock OCaml (`Domain.spawn`) or OxCaml (`Domain.Safe.spawn`)
Source	github.com/firstpair/phindex
Related projects	phidentify, tiffgraph, Semantic Unison, dng-compare